

Tuplas e Strings

Tuplas

- Uma **tuple** é uma estrutura *imutável* usada para representar um conjunto de itens de dados.
- O conjunto de itens de uma tupla é uma sequência de valores entre parênteses, separados por vírgula.
- Cada item possui uma posição, acessado por meio do índice.

```
>>> x = (1, 2, 3)    # uma tupla com três itens
>>> print(x)
      (1, 2, 3)
>>> x[0]
      1
>>> x[0] = 4        # Esse comando gera um erro. Qual o erro?
```

Tuplas

- Uma tupla vazia se escreve com parênteses vazios: ().
 - Parênteses são opcionais se não provocarem ambiguidade.
- Uma tupla com apenas um item é escrita com uma vírgula no final.

```
>>> x = (2)           # não é tupla, é o número inteiro 2
>>> x = (2,)         # uma tupla com um item (final com vírgula)
>>> 10,              # uma tupla com um item (final com vírgula)
    (10,)
>>> 3 * (10+3)        # não é tupla, é uma expressão aritmética
    39
>>> 3 * (10+3,)       # uma tupla com três itens
    (13, 13, 13)
```

A função *tuple*

- Além de parênteses (), podemos usar a função **tuple()** para construir uma tupla a partir de uma sequência qualquer.

```
>>> a = ()           # cria uma tupla vazia
>>> a = tuple("abcd") # cria uma tupla com quatro letras
>>> print(a)
    ('a', 'b', 'c', 'd')
>>> a = tuple([1,2,3]) # cria uma tupla a partir de uma lista
>>> print(a)
    (1, 2, 3)
>>> a = tuple(1234)    # Esse comando gera um erro. Qual o erro?
```

Sobrescrevendo uma tupla

- Embora não seja possível modificar uma tupla, podemos atribuir um novo valor a uma variável que armazena uma tupla.
 - Ou seja, alteramos a tupla toda. Exemplos:

```
>>> a = (1, 2, 3, 4)      # Cria uma tupla com quatro itens
```

```
>>> print(a)
(1, 2, 3, 4)
```

```
>>> a[0] = 5              # Esse comando gera um erro.
```

```
Traceback (most recent call last):
```

```
  File "<pyshell#10>", line 1, in <module> a[0] = 5
```

```
TypeError: 'tuple' object does not support item assignment
```

```
>>> a = (6, 7, 8)        # Altera a variável tupla
```

```
>>> print(a)             # Qual a saída desse comando?
```

Quando usar tuplas

- Em geral, tuplas podem ser substituídas com vantagem por listas.
 - Não é obrigatório o uso de tupla em Python.
- Entretanto, algumas construções em Python requerem tuplas ou sequências imutáveis, por exemplo:
 - Tuplas (ao contrário de listas) podem ser usadas como chaves de dicionários.
 - Funções com número variável de argumentos acessam os argumentos por meio de tuplas.
 - O operador de formatação % aceita tuplas, mas não listas.

O operador de formatação

- As **strings** suportam o operador % que, dada uma string especial (**template**) e um **valor**, produz uma string formatada.
- O formato geral é:
template % valor
- O **template** é uma string entremeada por **códigos de formatação**.
 - Um **código de formatação** é em geral composto do caracter % seguido de uma letra descritiva do tipo do valor a formatar.
 - Ex.: **s** para string, **f** para float, **d** para inteiro, dentre outros.

```
>>> '====%d====' % 100
'====100===='
```

```
>>> '====%f====' % 1
'====1.000000===='
```

Formatando tuplas

- Um **template** pode ser aplicado aos diversos valores de uma tupla para construir uma string formatada.

- Exemplo:

```
>>> template = "%s tem %d anos"
>>> tupla = ('Pedro', 10)
>>> print(template % tupla)
'Pedro tem 10 anos'
```

- Nas próximas aulas veremos que o operador de formatação também pode ser aplicado a dicionários.

Especificações de formato

- **Caracter:** %
- **Flags de conversão (opcionais):**
 - - indica alinhamento à esquerda.
 - + indica que um sinal deve preceder o valor convertido.
 - " " (um branco) indica que um espaço deve preceder números positivos.
 - 0 indica preenchimento à esquerda com zeros.
- **Comprimento mínimo do campo (opcional):**
 - O valor formatado terá este comprimento no mínimo.
 - Se igual a * (asterisco), o comprimento será lido da tupla.
- **Um "." (ponto) seguido pela precisão (opcional):**
 - Usado para converter as casas decimais de floats.
 - Se aplicado para strings, indica o comprimento máximo.
 - Se igual a *, o valor será lido da tupla.
- **Caracter indicador do tipo de formato**

Tipos de formato

- **d, i** Número inteiro escrito em decimal
- **o** Número inteiro sem sinal escrito em octal
- **u** Número inteiro sem sinal escrito em decimal
- **x** Número inteiro sem sinal escrito em hexadecimal (minúsculas)
- **X** Número inteiro sem sinal escrito em hexadecimal (maiúsculas)
- **e** Número de ponto flutuante escrito em notação científica ('e' minúsculo)
- **E** Número de ponto flutuante escrito em notação científica ('E' maiúsculo)
- **f, F** Número de ponto flutuante escrito em notação convencional
- **g** Mesmo que **e** se expoente é maior que -4. Caso contrário, igual a **f**
- **G** Mesmo que **E** se expoente é maior que -4. Caso contrário, igual a **F**
- **c** Caractere único (usado com inteiro ou string de tamanho 1)
- **r** String (entrada é qualquer objeto Python que é convertido usando a função repr)

Exemplos

```
>>> "Número inteiro: %d" % 55
'Numero inteiro: 55'
>>> "Numero inteiro com 3 casas: %3d" % 55
'Numero inteiro com 3 casas: 55'
>>> "Inteiro com 3 casas e zeros a esquerda: %03d" % 55
'Inteiro com 3 casas e zeros a esquerda: 055'
>>> "Inteiro escrito em hexadecimal: %x" % 55
'Inteiro escrito em hexadecimal: 37'
>>> from math import pi
>>> "Ponto flutuante: %f" % pi
'Ponto flutuante: 3.141593'
>>> "Ponto flutuante com 12 decimais: %.12f" % pi
'Ponto flutuante com 12 decimais: 3.141592653590'
>>> "Ponto flutuante com 10 caracteres: %10f" % pi
'Ponto flutuante com 10 caracteres: 3.141593'
>>> "Ponto flutuante em notacao cientifica: %10e" % pi
'Ponto flutuante em notacao cientifica: 3.141593e+00'
>>> "String com tamanho maximo definido: %.3s" % "Pedro"
'String com tamanho maximo definido: Ped'
```

Exemplo: Imprimindo uma tabela

```
itens = ["Abacate", "Limão", "Manga", "Melancia", "Laranja"]
precos = [7.13, 1.50, 4.95, 2.95, 9.00]
len_precos = 10          # Coluna de precos tem 10 caracteres
# Achar a largura da coluna de itens
len_itens = len(itens[0])
for item in itens:
    len_itens = max(len_itens, len(item))
# Imprimir tabela de precos
print("-"*(len_itens+len_precos))
print("%-*s%s" % (len_itens, "Item", len_precos, "Preço"))
print("-"*(len_itens+len_precos))
for i in range(len(itens)):
    print("%-*s%.2f" % (len_itens, itens[i], len_precos, precos[i]))
```

Exemplo: resultados

```
-----  
Item          Preço  
-----  
Abacate       7.13  
Limão         1.50  
Manga         4.95  
Melancia      2.95  
Laranja       9.00
```

O Módulo String

- Manipulação de strings é uma atividade frequente em programas Python.
- Existe um módulo chamado **string** que contém uma grande quantidade de funcionalidades para trabalhar com strings.
 - Para usá-las temos que importar o pacote:

```
from string import *
```

- Entretanto, strings pertencem à classe **str** e a maior parte do que existe no módulo **string** aparece como métodos da classe **str**.

String: método find

- ADI? (**substring**, **início**, **fim**)
 - Retorna o índice da primeira ocorrência de **substring**.
 - **início** e **fim** são opcionais e indicam o intervalo de índices onde a busca será efetuada.
 - Por default **início** = 0 e **fim** = **len(string)**, respectivamente.
 - Caso **substring** não apareça na string, ADI? , „ retorna -1.
 - O operador **DI** pode ser usado para dizer se uma substring aparece em uma string.

String: método find

```
>>> s = "quem parte e reparte, fica com a maior parte"
>>> s.find("parte")
5
>>> s.find("reparte")
13
>>> s.find("parcela")
-1
>>> "parte" in s
True
>>> s.find("parte", 6)
15
>>> s.find("parte", 6, 12)
-1
```

String: método join

- **join (sequência)**

- Retorna uma string com todos os elementos da **sequência** concatenados. Os elementos da sequência devem ser strings.
- A string objeto é usada como separador entre os elementos.
Exemplos:

```
>>> "/" .join(("usr", "bin", "python"))
'usr/bin/python'
```

```
>>> "/" .join("usr bin python")
'u/s/r/ /b/i/n/ /p/y/t/h/o/n'
```

```
>>> "/" .join("usr", "bin", "python")
TypeError: str.join() takes exactly one argument (3 given)
```

String: método join

- **join (sequência)**

- Mais exemplos:

```
>>> "Q".join((1, 2, 3, 4, 5))
TypeError: sequence item 0: expected str instance, int found
```

```
>>> "Q".join(('1', '2', '3', '4', '5'))
'1Q2Q3Q4Q5'
```

String: métodos **lower** / **casefold**

- **lower()**

- Retorna uma string com todos os caracteres maiúsculos convertidos para minúsculos.

- Veja também o método **casefold()**.

```
>>> lp = "PYTHON"
>>> lp = lp.lower()      # ou lp = lp.casefold()
>>> print(lp)
python

>>> print("Lógica de Programação".lower())
lógica de programação
```

String: método **upper**

- **upper()**

- Retorna uma string com todos os caracteres minúsculos convertidos para maiúsculos.

```
>>> lp = "python"
>>> lp = lp.upper()
>>> print(lp)
PYTHON

>>> print("Lógica de Programação".upper())
LÓGICA DE PROGRAMAÇÃO
```

String: método **replace**

- **replace(velho, novo, n)**

- Substitui, na saída, as instâncias da substring **velho** por **novo**.
- Se **n** for especificado, apenas **n** instâncias são trocadas.
- Caso contrário, todas as instâncias são trocadas.
 - O método **replace** não altera o conteúdo da string.

```
>>> s = "quem parte e reparte, fica com a maior parte"
>>> s.replace("parte","parcela")
'quem parcela e reparcela, fica com a maior parcela'
>>> s = "quem parte e reparte, fica com a maior parte"
>>> s.replace("parte","parcela", 2)
'quem parcela e reparcela, fica com a maior parte'
```

String: método **split**

- **split(separador)**

- Retorna uma lista (**list**) com as substrings presentes entre cópias da string **separador**.
- Faz o contrário do método **join**.
- Se **separador** não for especificado, é assumido sequências de caracteres em branco (espaços), tabs ou newlines.

```
>>> s = "xxx yyy zzz xxx yyy zzz"
>>> s.split()
['xxx', 'yyy', 'zzz', 'xxx', 'yyy', 'zzz']
>>> s.split('xxx')
['', ' yyy zzz ', ' yyy zzz']
```

String: método **strip**

- **strip(ch)**

- Retorna a string sem caracteres iniciais ou finais que estejam na string **ch**.
- Se **ch** não for especificada, retira caracteres em branco.
- Pode-se também usar **rstrip()** para retirar caracteres à direita (final) ou **lstrip()** para retirar caracteres à esquerda (início).

```
>>> " xxx afdafa ".strip()
'xxx afdafa'
>>> "xxx yyy zzz xxx".strip("xy ")
'zzz'
>>> " xxx ".rstrip()
' xxx'
```

String: método **translate**

- **translate(trans)**

- Retorna uma cópia da string onde os caracteres são substituídos de acordo com a tabela de tradução **trans**.
- **trans** é uma string com 256 caracteres, um para cada possível código de oito bits.
 - Ex.: se **trans** tem 'X' na posição 65 (correspondente ao caractere ASCII 'A'), então, na string retornada, todos os caracteres 'A' terão sido substituídos por 'X'
- Na verdade, as tabelas de tradução são normalmente construídas com a função **maketrans** do módulo **string**.

Função `string.maketrans`

- `maketrans(velho, novo)`
 - Retorna uma tabela de tradução onde os caracteres em **velho** são substituídos pelos caracteres em **novo**.

```
>>> from string import maketrans
>>> trans = maketrans('qs', 'kz')
>>> s = "que surpresa: quebrei a cara"
>>> s.translate(trans)
'kue zurpreza: kuebrei a cara'
```